

KNN, SVMs, Tree Models, Ensembles, Tuning

Sattam Altwaim

sattam.tuuaim@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

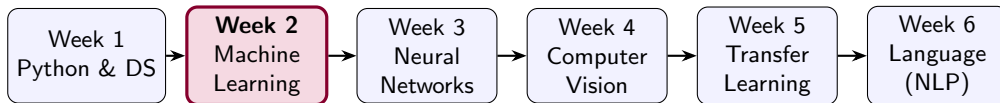


موهبة
Mawhiba

KAUST Academy & Mawhiba
AI Olympiad Training (Beginners)
Week 2, Day 10, Lecture

28 September 2026

1. Big Picture and Motivation
2. Learning Outcomes
3. k-Nearest Neighbours
4. Support Vector Machines
5. Decision Trees
6. Ensembles
7. Tuning and Feature Engineering
8. Olympiad Practice
9. Limitations and What Comes Next
10. Summary
11. References



Today: Day 10 of 42

- ▶ **Saturday:** one model family, the line (and its sigmoid twin). **Yesterday:** how to judge any model honestly: metrics, validation, the overfitting alarm, regularization.
- ▶ **Today:** the rest of the classic toolbox. Four new model families, each with one idea, one dial and one weakness, and the recipe for tuning their dials.
- ▶ **This week's finish:** Days 11 to 13 deepen each family; every tabular task from now on starts with the models of today.

So far, every model draws a **straight line** (or a flat plane) through the data.

Task	Best linear model so far	What the data actually looks like
used-car prices	MAE 20,218 SAR, and only after log price and 69 columns	price drops fast in the first 100,000 km, then flattens
Tahoe mileage curve	a line: MAE 28,054; degree 3 by hand: 23,171	a curve whose shape we had to guess
admissions	logistic regression, ROC-AUC 0.950	Budapest matters only when the advanced maths exam was not taken

The problem

A line adds up the columns. It cannot say “if this, then look at that”, it cannot bend on its own, and to make it bend we had to invent polynomial columns by hand and then tame them with Ridge.

In one sentence

Instead of one formula for the whole table, let the model **look up neighbours**, **draw the widest possible gap**, or **ask a sequence of questions**, and then **combine many such models** so their mistakes cancel.

Family	The one idea	The main dial
k-nearest neighbours	answer like your k closest rows	k
support vector machine	the boundary with the widest margin	C , the kernel
decision tree	yes/no questions until the groups are pure	<code>max_depth</code>
random forest, boosting	hundreds of trees voting or correcting each other	<code>n_estimators</code> , <code>depth</code>

Where you will meet it

On almost every tabular Olympiad task the winning notebook is a tuned random forest or gradient boosting on well-engineered columns. Today you learn why, and how to tune them without fooling yourself.

By the end of this session, you will be able to:

- ▶ **Explain** how k-nearest neighbours, support vector machines, decision trees, random forests and gradient boosting make a prediction.
- ▶ **Compute** a Gini impurity and pick the better of two splits by hand.
- ▶ **Name** the main dial of each model and predict what turning it does to the training and validation errors.
- ▶ **Read** a random forest's feature importances and a decision tree's rules.
- ▶ **Tune** hyperparameters with `GridSearchCV` and cross-validation, and compare models by the task's metric.
- ▶ **Build** new columns (feature engineering) and say which models need scaling and which do not.

Classic Models: **k-Nearest Neighbours**

Definition

k-nearest neighbours (KNN) stores the whole training table. To predict a new row it finds the k training rows closest to it (Day 02's distance) and answers with their majority label (classification) or their mean target (regression).

- ▶ Nothing is learned in `fit`: the data *is* the model. Prediction is where the work happens.
- ▶ Distance between two students with columns (math, informatics):
$$\sqrt{(87 - 78)^2 + (91 - 51)^2} = \sqrt{81 + 1600} = 41.0.$$

Worked example with k

A new student's three closest rows have labels (admitted, admitted, not admitted). Majority: **admitted**, with probability $2/3$. With $k = 5$ and labels (1, 1, 0, 0, 0) the answer flips to not admitted. The dial changes the answer.

Day 02's nearest-centroid model was KNN with one "neighbour" per class: the class mean.

Distance adds up squared differences, so a column with big numbers decides everything.

column	range	max effect
math	12 to 100	$88^2 = 7744$
grade_12	2 to 5	$3^2 = 9$
county_Budapest	0 or 1	1

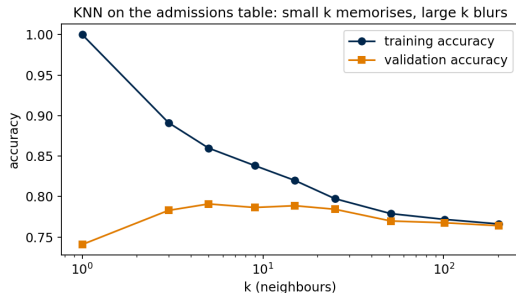
Without scaling, grades and county are invisible to KNN.

On the admissions table, $k = 15$

unscaled columns: validation accuracy **0.699**, below the 0.705 majority baseline.

after StandardScaler: **0.789**.

- ▶ Same rule as logistic regression and Ridge:
`make_pipeline(StandardScaler(), KNeighborsClassifier(k)).`
- ▶ Trees and forests (next sections) do not care about scale: they only ask “is x above a cut?”.



Source: HAIO 2025 admissions data, 54 scaled columns, 80/20 split;
scripts/day10_models_figures.py.

- ▶ $k = 1$: training 1.0 (each row is its own neighbour), validation 0.741: overfitting.
- ▶ $k = 5$ to 15: validation peaks near 0.79. $k = 201$: both sink to 0.76.

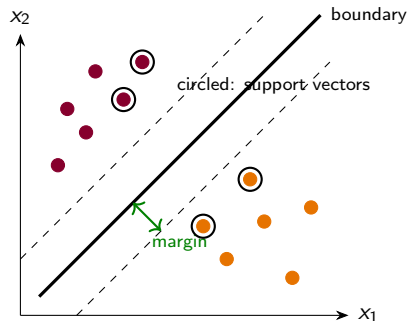
```
from sklearn.neighbors import  
    KNeighborsClassifier  
knn = make_pipeline(StandardScaler(),  
    KNeighborsClassifier(  
        n_neighbors=15))  
knn.fit(X_train, y_train)  
knn.predict_proba(X_val)[: , 1] # share  
    of 15
```

Classic Models: **Support Vector Machines**

Definition

A **support vector machine (SVM)** draws the separating boundary that leaves the **widest margin** (empty street) between the two classes. The few points that touch the street are the **support vectors**; they alone fix the boundary.

- ▶ Logistic regression also draws a line, but any line that separates the training rows will do. The SVM picks the one farthest from both classes, which generalises better.
- ▶ Dial C: how much a point inside the street (or on the wrong side) costs. Small C: wide street, some mistakes allowed. Large C: narrow street, every training point obeyed, overfitting.



When no straight line separates the classes, a **kernel** lets the SVM measure similarity in a way that bends the boundary: the **RBF kernel** treats two rows as similar when they are close, like KNN, and the boundary becomes a smooth curve around each class.

- ▶ `kernel="linear"`: a straight boundary, fast, readable weights.
- ▶ `kernel="rbf"` (the default): curved boundaries; a second dial, `gamma`, sets how local the curve is.
- ▶ Needs scaled columns, like KNN. Slow above about 50,000 rows.

Skipped on purpose

The mathematics of the margin and of kernels is Day 11 (and Level 2). Today: the picture and the two dials.

```
from sklearn.svm import SVC
svm = make_pipeline(StandardScaler(),
                    SVC(kernel="rbf",
                        C=1.0,
                        probability=
                            True))

svm.fit(X_train, y_train)
```

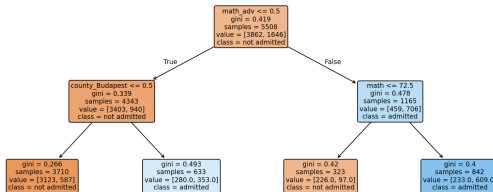
On the admissions table

5-fold ROC-AUC 0.942, accuracy 0.863. Close to logistic regression (0.950), with no feature invented by hand. `probability=True` is needed for `predict_proba`.

Classic Models: **Decision Trees**

Definition

A **decision tree** predicts by asking yes/no questions about one column at a time (“is math above 72.5?”), following the answers down to a **leaf**, and answering with the majority label (or mean target) of the training rows that landed in that leaf.



Source: HAIIO 2025 admissions data, DecisionTreeClassifier(max_depth=2) drawn with `sklearn.tree.plot_tree`; scripts/day10_models_figures.py.

Left branch: no advanced maths exam, not from Budapest: 3,710 students, 587 admitted, the leaf says “not admitted”. Rules a person can check, no scaling, and “if this then that” for free.

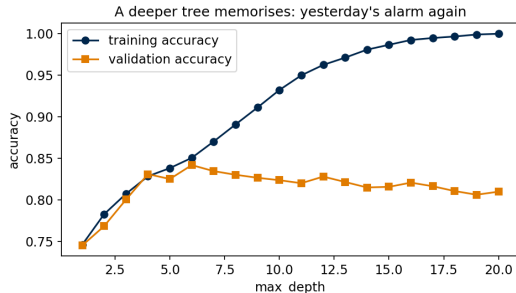
$G = 1 - \sum_c p_c^2$, where p_c is the share of class c in the node: $G = 0$ is pure, 0.5 is a 50/50 mix.

The root of the tree above

5,508 students, 3,862 not admitted, 1,646 admitted: $G = 1 - (0.701^2 + 0.299^2) = 1 - (0.491 + 0.089) = 0.419$. Split on `math_adv`: left [3403, 940], $G = 0.339$; right [459, 706], $G = 0.478$; weighted $(4343 \cdot 0.339 + 1165 \cdot 0.478)/5508 = 0.368$, an improvement of 0.051.

candidate question at the root	weighted G after the split	improvement
<code>math_adv</code> $\leq$ 0.5	0.368	0.051
<code>county</code> is Budapest	0.380	0.039
<code>math</code> $\leq$ 72.5	0.402	0.017
<code>grade_12</code> $\leq$ 3.9	0.417	0.002

The tree tries every column and every cut, keeps the best, then repeats inside each child. No gradient descent: greedy search.



Source: admissions data, depths 1 to 20, 80/20 split; scripts/day10_models_figures.py.

- ▶ Depth 20: training accuracy 0.999, validation 0.810. The tree has one leaf per student: pure memory.
- ▶ Depth 6: validation 0.842, the best. Yesterday's alarm, with a new dial.
- ▶ Trees are also unstable: change a few rows and the root question changes.

```
from sklearn.tree import  
DecisionTreeClassifier  
tree = DecisionTreeClassifier(max_depth  
=6, random_state=0)  
tree.fit(X_train, y_train)           # no  
scaler needed
```

Classic Models: **Ensembles: Forests and Boosting**

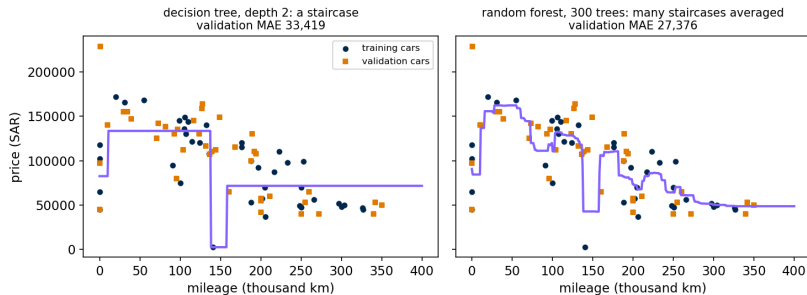
Definition

A **random forest** trains hundreds of decision trees, each on a random sample of the rows (drawn with replacement) and a random subset of the columns at every split, and predicts by **majority vote** (classification) or by **averaging** (regression).

- ▶ One deep tree overfits, but different trees overfit *differently*. Averaging 300 of them keeps what they agree on (the pattern) and cancels what they disagree on (the noise).
- ▶ The randomness is the point: identical trees would vote identically and average nothing.
- ▶ Dials: `n_estimators` (more trees never hurt, only cost time) and `max_depth` (often left unlimited).

On the admissions table

one tree, depth 5: 5-fold ROC-AUC 0.891. Forest of 300 trees: **0.935**, spread 0.003.
On the used cars: one tree MAE 23,034; forest **15,250**; the best linear model 20,218.



Source: the 80 Chevrolet Tahoe listings of Day 09, mileage to price; `scripts/day10_models_figures.py`.

- ▶ A tree predicts a **staircase**: one constant per leaf. It never extrapolates: past the last training car every prediction is the last step.
- ▶ The forest averages 300 staircases with different steps: a smoother curve that a line could not draw and nobody had to guess the shape of.

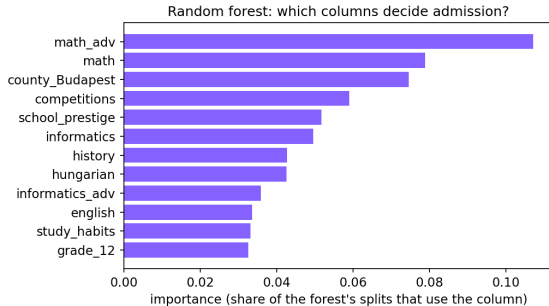
Definition

Gradient boosting builds small trees one after another; each new tree is fitted to the **errors** the previous trees still make, and its prediction is added with a small weight (the learning rate).

- ▶ Forest: independent trees, averaged. Boosting: a chain, each tree correcting the sum so far. Day 08's gradient descent with a tree as the step.
- ▶ Dials: `n_estimators`, `learning_rate`, `max_depth` (usually 2 to 4). Too many steps with a big rate overfits, like a big α on Day 08.
- ▶ Usually the strongest tabular model; `LightGBM` and `XGBoost` are fast versions (Day 12).

```
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
rf = RandomForestClassifier(n_estimators=300, random_state=0).fit(X_train, y_train)
gb = GradientBoostingClassifier(n_estimators=200, max_depth=3).fit(X_train, y_train)
```

Admissions, 5-fold ROC-AUC: forest 0.935, boosting **0.944**. Used cars, MAE: forest **15,250**, boosting 16,416.



Source: admissions data, `RandomForestClassifier(300).feature_importances_`, top 12 of 54 columns; `scripts/day10_models_figures.py`.

Definition

A column's **feature importance** is how much of the forest's total impurity reduction happened on splits of that column. The importances of all columns add to 1.

- ▶ `math_adv` 0.107, `math` 0.079, Budapest 0.075, `competitions` 0.059: the same story as Lab 3's logistic weights, without scaling.
- ▶ Use it to drop useless columns, to explain a model, and to guess which new column to build. Day 13 adds the caveats.

Classic Models: **Tuning and Feature Engineering**

Definition

A **parameter** is learned by fit (a weight, a split). A **hyperparameter** is a dial you set before fit (k , C , max_depth , n_estimators , α). **Tuning** means choosing the dials by cross-validation, never by the training score.

```
from sklearn.model_selection import
    GridSearchCV
grid = GridSearchCV(
    RandomForestClassifier(random_state=0)
    ,
    {"n_estimators": [100, 300],
     "max_depth": [4, 8, None]},
    cv=5, scoring="roc_auc")
grid.fit(X, y)
print(grid.best_params_, grid.best_score_)
model = grid.best_estimator_    # refitted
    on all rows
```

trees	depth	CV ROC-AUC
100	4	0.912
300	4	0.915
100	8	0.928
300	8	0.930
100	none	0.933
300	none	0.935

Six combinations, five folds each: 30 fits.
Use the task's metric in scoring.

No Free Lunch: Compare on the Task's Metric

Model	scaling	admissions ROC-AUC	used cars MAE	readable?
logistic / linear regression	yes	0.950	20,218	weights
KNN	yes	0.851	20,137	no
SVM, RBF kernel	yes	0.942	(too slow)	no
decision tree	no	0.891	23,034	the rules
random forest	no	0.935	15,250	importances
gradient boosting	no	0.944	16,416	importances

- ▶ The linear model wins the admissions task; the forest wins the cars by a quarter. **No model wins everywhere**: the shape of the data decides.
- ▶ So the Olympiad habit is: a quick baseline (linear), then a forest and boosting, tuned by cross-validation on the judge's metric; keep whichever scores best on rows it never saw.

Definition

Feature engineering is creating new input columns from the raw ones, so that the pattern becomes something the model can express: a number from a text, a ratio, a difference, a flag, a log.

Raw column	Engineered column	Why it helped
owners "20000 - 50000" (Day 08, Lab 2)	midpoint 35,000	a number the model can use at all
Release date	release year	prices rise with the year: MAE 7.84 to 7.01
Positive, Negative	<code>log1p(Positive)</code> , positive share	tamed a 0 to 1,000,000 spread: 6.89 to 6.77
exam score -1 (Day 08, Lab 3)	<code>_taken</code> flag plus 0	"not taken" became explicit
Year (cars)	<code>age = 2022 - Year</code> , <code>log Mileage</code>	straighter for a line; trees do not care

- ▶ Linear models need it most: they cannot bend or combine columns. Trees find cuts and interactions, but cannot invent a ratio or read a date.
- ▶ Build the column in one function applied to train and test alike (Day 08's `add_features`).

Classic Models: **Olympiad Practice**

Task

Each row is one heartbeat from an ECG: 150 numbers (the voltage over time). Five classes: normal, and four disorders. Training set 2,000 beats, of which 1,400 are normal; validation set 1,500. The classifier must run on a tiny device, so the rules are strict: compute **at most 4 numbers** per beat with NumPy or SciPy, then train a `RandomForestClassifier` with **at most 10 trees of depth at most 10**. Nothing else is allowed (no SVM, no neural network). Score: **balanced accuracy** on a hidden test set (the mean of the five per-class recalls). 0 points below 0.75, full points at 0.98, linear in between.

- ▶ **Question 1:** why balanced accuracy and not accuracy? What does “always normal” score on each?
- ▶ **Question 2:** the organisers’ example uses the min, max, mean and standard deviation of the 150 values. With 3 trees it reaches balanced accuracy 0.45 on the validation set; with 10 trees, 0.46. Where do the points have to come from?

Adapted from II Olimpiada AI (Poland, 2025), stage 1, task 3 “Wykrywanie zaburzeń sygnału EKG”; data and statement in the OlimpiadaAI GitHub repository.

1. **Recognise:** 70% one class. Accuracy gives “always normal” 0.55; balanced accuracy gives it 0.20 (one class right out of five). The metric forces the rare disorders to be found.
2. **The model is fixed and tiny**, so the task is pure **feature engineering**. Min, max, mean and std ignore *where* things happen; a doctor reads where the spike is, how wide it is, how high the segment after it sits.
3. **Loop like an Olympian:** feature function, 10-tree forest, validation balanced accuracy, confusion matrix, then the next feature for the class still missed.

```
def features(X):
    amp = X.max(axis=1) - X.min(axis=1)          # X has shape (n, 150)
    where = X.argmax(axis=1)                     # height of the spike
    after = X[:, 90:120].mean(axis=1) - X[:, :30].mean(axis=1) # position of the spike
    width = (X > 0.5 * X.max(axis=1, keepdims=True)).sum(axis=1) # level after
    return np.c_[amp, where, after, width]        # spike width

rf = RandomForestClassifier(n_estimators=10, max_depth=10, random_state=42)
rf.fit(features(X_train), y_train)
print(balanced_accuracy_score(y_val, rf.predict(features(X_val))))
```

Check: beat the example's 0.46; points begin at 0.75; judge features on validation, never on training.

Classic Models: **Limitations & What Comes Next**

- ▶ **KNN** keeps every row and computes every distance at prediction time: slow on big tables, and lost in many columns (every point is far from every other).
- ▶ **SVMs** scale badly past tens of thousands of rows, and the RBF boundary cannot be read.
- ▶ **Trees and forests never extrapolate**: a car newer than every training car gets the price of the newest training car. Lines do extrapolate (sometimes wrongly).
- ▶ **Boosting overfits quietly** when `n_estimators` is large and the learning rate high; always watch the validation curve.
- ▶ **Grid search is expensive**: 6 combinations took 30 fits; 4 dials with 5 values each take 3,125. Day 12 shows random search.

The common mistake

Tuning on the same rows used to report the final score. Grid search chooses on the folds; the final number comes from a split the search never touched.

- ▶ **Day 11, self-study:** linear and logistic regression in depth, the SVM margin mathematics, regularization dials (C , α).
- ▶ **Day 12, self-study:** trees, forests and boosting in depth (LightGBM, XGBoost), feature engineering recipes for tables.
- ▶ **Day 13, self-study:** evaluation habits, cross-validation strategies, PCA, feature importance and its traps, the checklist for Friday.
- ▶ **Week 3:** the model family that builds its own features: neural networks, trained by the same gradient descent as Day 08.

Classic Models: **Summary**

- ▶ **KNN** answers like the k closest rows; needs scaling; small k memorises, large k blurs.
- ▶ **SVM** draws the widest-margin boundary; C trades margin for mistakes, the RBF kernel bends it; needs scaling.
- ▶ **Decision tree**: yes/no questions chosen by Gini impurity; readable rules; `max_depth` is the overfitting dial; no scaling.
- ▶ **Random forest** averages hundreds of random trees; **gradient boosting** chains trees that fix each other's errors; both give **feature importances**.
- ▶ **Tuning**: hyperparameters are set by `GridSearchCV` on the task's metric; no model wins every table, so compare on validation rows.
- ▶ **Feature engineering**: build the column that makes the pattern expressible, in one function for train and test.

Classic Models: **References & Resources**

- [1] Hungarian AI Olympiad (2025). *Online qualifier: University Admissions* (CC BY-NC-SA 4.0). Used for the decision-region, KNN, tree, importance and comparison figures. <https://github.com/Hungarian-AI-Olympiad/HAIO-Hungarian-AI-Olympiad/tree/main/2025/nyari-online>
- [2] Bin Talib, T. *Saudi Arabia Used Cars Dataset*, Kaggle [turkibintalib/saudi-arabia-used-cars-dataset](https://www.kaggle.com/datasets/turkibintalib/saudi-arabia-used-cars-dataset). Used for the regression comparison and the staircase figure. <https://www.kaggle.com/datasets/turkibintalib/saudi-arabia-used-cars-dataset>
- [3] Olimpiada AI (2025). *II Olimpiada Sztucznej Inteligencji, etap 1, zadanie 3: Wykrywanie zaburzeń sygnału EKG*. Adapted for the practice task. https://github.com/OlimpiadaAI/II-OlimpiadaAI/tree/main/1_etap
- [4] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR* 12. neighbors, svm, tree, ensemble, GridSearchCV. <https://scikit-learn.org>
- [5] Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, 3rd ed., chapters 5 to 7: SVMs, decision trees, ensembles.
- [6] All plots: `slides-builder/scripts/day10_models_figures.py` (matplotlib, real data only).